

72.07 - Protocolos de Comunicación

Proyecto Especial

VERSION PROTOTIPAL



Integrantes:

- Asin, Gerónimo - 65486
- Garbarz, Andrés Matías - 65488
- Lee, Sebastián Nicolás - 64675
- Lifischtz, Santiago - 65782

Grupo: 16

Fecha de entrega: 07/07/2026

Índice

Índice.....	1
1. Abstracto.....	3
2. Protocolos y Aplicaciones.....	3
2.1 Servidor Proxy SOCKSv5.....	3
2.2 Protocolo de Monitoreo - ChungusMonitor.....	5
2.2.1 Transporte y diseño general.....	5
2.2.2 Framing.....	5
2.2.3 Establecimiento de la sesión.....	5
2.2.4 Autenticación.....	6
2.2.5 Comandos.....	7
Los errores utilizan mensajes específicos:.....	8
2.2.6 Parámetros configurables.....	8
2.2.7 Pipelining.....	9
2.2.8 Cierre de la conexión.....	9
2.2.9 Límites propuestos.....	9
2.3 Aplicación Cliente.....	10
3. Proceso de Desarrollo.....	10
3.1 Echo Server Bloqueante.....	10
3.2 Echo Server No Bloqueante.....	11
3.3 Implementación de SOCKSv5.....	11
3.4 Desarrollo del Protocolo de Administración y Aplicación Cliente.....	11
4. Problemas Encontrados.....	12
5. Limitaciones.....	13
6. Posibles Extensiones.....	14
7. Conclusiones.....	15
8. Guía de Instalación.....	16
8.1 Requisitos del Entorno.....	16
8.2 Obtención del Código y Compilación.....	16
8.3 Entorno Docker.....	17
8.4 Artefactos Generados.....	17
9. Guía de Configuración.....	18
9.1 Argumentos del Servidor.....	18
9.2 Argumentos del Cliente de Monitoreo.....	18

9.3 Parámetros en Tiempo de Ejecución.....	19
9.4 Usuario Inicial y Roles.....	19
10. Ejemplos de Prueba.....	20
10.1 Pruebas Automatizadas.....	20
10.2 Pruebas Funcionales del Proxy SOCKS.....	21
Destino por FQDN (resolución DNS en el servidor).....	21
Destino por FQDN resuelto por curl (el proxy recibe la IP).....	21
Credenciales inválidas: curl reporta el rechazo de autenticación.....	21
Destino inexistente: el proxy responde host inalcanzable (REP 0x04).....	21
Puerto cerrado: el proxy responde conexión rechazada (REP 0x05).....	21
10.3 Pruebas de Estrés y Throughput.....	22
11. Ejemplos de Configuración y Monitoreo.....	23
Gestion de usuarios.....	23
Registro de Accesos y Sesiones Activas.....	24
Cambios de Configuración en Tiempo de Ejecución.....	24
12. Documento de Diseño del Proyecto.....	25
13. Bibliografía.....	26

1. Abstracto

El presente documento detalla el proceso de desarrollo de un servidor proxy que implementa el protocolo SOCKSv5, capaz de atender a todos sus clientes desde un único hilo mediante E/S no bloqueante multiplexada, dentro del marco de la materia Protocolos de Comunicación en el Instituto Tecnológico de Buenos Aires. Además, se implementó un protocolo de administración custom (ChungusMonitor) que permite monitorear y configurar el proxy en tiempo de ejecución.

2. Protocolos y Aplicaciones

A continuación se describen las aplicaciones y protocolos desarrollados para el presente trabajo, con sus funcionalidades y descripciones correspondientes. Este consta de un servidor que implementa el protocolo SOCKSv5 sobre el puerto 1080 y el protocolo ChungusMonitor sobre el puerto 8080 (ambos configurables), así como una aplicación cliente destinada al monitoreo. Ambos servicios se ejecutan dentro de un mismo proceso, compartiendo un único selector de entrada/salida y un único almacén de estado, lo que garantiza que la información expuesta a través del puerto de monitoreo refleje en todo momento el estado real del proxy.

2.1 Servidor Proxy SOCKSv5

El servidor al inicializarse, crea un socket pasivo IPv6 dual-stack, capaz de aceptar tanto conexiones IPv4 como IPv6 sobre un mismo descriptor. Para ello se desactiva la opción IPV6_V6ONLY y se habilita SO_REUSEADDR, tras lo cual el socket se registra en el selector con interés de lectura para comenzar a aceptar nuevas conexiones. Cada cliente aceptado se representa mediante una sesión que contiene dos buffers cruzados de 4096 bytes **c2o** (cliente hacia origen) y **o2c** (origen hacia cliente) junto con una máquina de estados que modela las distintas etapas del protocolo.

La primera etapa corresponde a la negociación del método de autenticación. Un parser consume el mensaje inicial del cliente byte a byte y verifica los métodos ofrecidos. Si entre ellos se encuentra el método usuario/contraseña (**0x02**), el servidor responde con los bytes **[05][02]** y continúa hacia la fase de autenticación. Si el cliente no ofrece (**0x02**) pero sí el método sin autenticación (**0x00**), el servidor acepta y procede a registrar al usuario como anónimo (username: anon). En caso contrario, responde **[05][FF]** y cierra la conexión una vez enviada la respuesta. GSSAPI no se implementa, como especifica la consigna.

La autenticación se implementa conforme al RFC 1929. En esta etapa, un segundo parser interpreta la subnegociación de usuario y contraseña y valida las credenciales contra la tabla de usuarios del **monitor_store**, utilizada también por el protocolo de monitoreo. Cuando las credenciales son válidas, el servidor responde **[01][00]**; si son inválidas o el mensaje recibido está mal formado, responde **[01][01]** y finaliza la conexión.

Superada la autenticación, el servidor procesa el pedido de conexión (**CONNECT**) mediante un tercer parser encargado de interpretar el request definido en el RFC 1928. Se soportan los tres tipos de dirección especificados por el protocolo (IPv4, IPv6 y FQDN), aunque únicamente se acepta el comando **CONNECT**, rechazándose los comandos **BIND** y **UDP ASSOCIATE**.

Cuando el destino se especifica mediante un nombre de dominio (FQDN), la resolución DNS se realiza mediante la llamada bloqueante **getaddrinfo(3)** ejecutada en un hilo dedicado, creado exclusivamente para esa operación y descartado una vez finalizada. Al completar la resolución, dicho hilo despierta al selector mediante **selector_notify_block()**, permitiendo que la máquina de estados continúe su ejecución en el hilo principal. Éste constituye el único uso de hilos adicionales dentro de todo el servidor.

La conexión con el servidor de origen se establece mediante un **connect(2)** no bloqueante. El descriptor correspondiente se registra con interés de escritura y es el kernel quien notifica posteriormente si la conexión se completó o falló, información que se obtiene mediante **getsockopt(SO_ERROR)**. Si la resolución DNS produjo múltiples direcciones y la conexión falla, ya sea de manera inmediata o asincrónica, el servidor reintenta automáticamente utilizando la siguiente entrada de la lista de **addrinfo**; por ejemplo, si una dirección IPv6 no resulta accesible, se intenta posteriormente con una dirección IPv4.

Una vez determinado el resultado del intento de conexión, el proxy responde al cliente utilizando los códigos de estado definidos por el protocolo. Se devuelve **0x00** cuando la conexión se establece correctamente, **0x01** ante fallos generales del servidor, como mensajes mal formados o falta de recursos, **0x04** cuando la resolución DNS no logra alcanzar el host solicitado y **0x05** cuando todas las direcciones obtenidas durante la resolución fallan al intentar establecer la conexión.

Luego de enviar esta respuesta, y siempre que la conexión haya sido exitosa, el servidor entra en la etapa de relay, donde realiza la copia bidireccional de datos entre cliente y origen. Los bytes leídos desde el cliente se almacenan en el buffer **c2o** y posteriormente se envían al servidor de origen, mientras que los datos recibidos desde el origen siguen el camino inverso a través del buffer **o2c**. Los intereses de lectura y escritura de ambos sockets se recalculan dinámicamente según exista espacio disponible o datos pendientes en cada buffer, implementando de manera

natural el control de flujo. En consecuencia, si el servidor de origen deja de consumir datos, el proxy deja temporalmente de leer información proveniente del cliente.

Durante todo el ciclo de vida de la sesión, cada transición de estado se notifica al **monitor_store**, incluyendo la aceptación de la conexión, la autenticación del usuario, el establecimiento de la conexión con el destino, la cantidad de bytes transferidos y el cierre o fallo de la sesión. Esta información alimenta las métricas del sistema, la lista de conexiones activas y el registro de accesos expuesto por el protocolo de monitoreo.

2.2 Protocolo de Monitoreo - ChungusMonitor

Esta subsección especifica el protocolo diseñado para el monitoreo y la configuración del servidor. La descripción es agnóstica al lenguaje de implementación y contiene los detalles necesarios para implementar un cliente o servidor compatible.

2.2.1 Transporte y diseño general

ChungusMonitor es un protocolo **de texto**, orientado a conexión, que corre sobre **TCP**. El servidor escucha en un puerto dedicado (8080 por defecto) distinto al del servicio SOCKS. Se eligió un formato de texto línea a línea — en la línea de protocolos como POP3 o el protocolo de Redis — por su simplicidad de implementación y depuración: cualquier problema puede diagnosticarse observando el tráfico directamente, y no se requiere serialización binaria ni consideraciones de endianness.

2.2.2 Framing

Cada mensaje, ya sea un comando o respuesta, es una línea de texto terminada en `\n`. El servidor también acepta el terminador `\r\n`, descartando el `\r`. La longitud máxima de una línea de comando es de **512 bytes**. Si un cliente la excede, el servidor responde: **-ERR line too long** y descarta lo acumulado. Los argumentos de un comando se separan con espacios simples, permitiendo hasta 8 tokens por comando.

2.2.3 Establecimiento de la sesión

Al aceptar una conexión, el servidor envía inmediatamente el saludo:

```
+OK ChungusMonitor v1.0
```

Toda respuesta del servidor comienza con **+OK** (éxito) o **-ERR** (error) seguido de texto descriptivo. Un comando puede producir múltiples líneas de respuesta (por ejemplo **CONNECTIONS** emite una línea **+OK** por sesión activa).

También, todas las respuestas, salvo el saludo inicial, finalizan con un línea que contiene únicamente un punto (.), actuando como delimitador para el fin de la respuesta. Esto permite al cliente distinguir el final de una respuesta multilínea como el HELP.

2.2.4 Autenticación

La sesión atraviesa una máquina de estados de dos fases:

En el estado inicial solo se aceptan los comandos **AUTH** y **HELP**. Cualquier otro comando produce el error: **-ERR not authenticated**.

La sentencia: **AUTH *usuario contraseña*** valida las credenciales contra la tabla de usuarios compartida con **SOCKS** y exige que el usuario tenga **rol administrador**.

Las respuestas posibles son:

- **+OK** (éxito)
- **-ERR invalid credentials** (credenciales incorrectas)
- **-ERR not admin** (credenciales válidas pero sin rol de administrador).

Una vez autenticado, un **AUTH** repetido produce **-ERR already authenticated**.

La decisión de autenticar dentro del propio protocolo (en lugar de, por ejemplo, restringir por dirección IP) reutiliza la tabla de usuarios existente y permite distinguir roles: los usuarios comunes pueden usar el proxy SOCKS pero no administrar el servidor.

2.2.5 Comandos

Comando	Sintaxis esperada	Descripción
AUTH	AUTH usuario contraseña	Autentica la sesión como administrador. Obligatorio antes de cualquier otro comando salvo HELP.
STATS	STATS	Devuelve las métricas globales.
CONNECTIONS	CONNECTIONS	Lista las sesiones SOCKS activas.
USERS	USERS	Lista todos los usuarios registrados en el servidor, indicando su rol (admin o user).
CONFIG	CONFIG parámetro valor	Modifica un parámetro de configuración en tiempo de ejecución.
ACCESS_LOG	ACCESS_LOG [usuario]	Devuelve el registro de accesos, del más reciente al más antiguo, opcionalmente filtrado por usuario.
ADD_USER	ADD_USER usuario contraseña [admin]	Crea un usuario; el sufijo admin le otorga rol administrador.
DEL_USER	DEL_USER usuario	Elimina un usuario. Se rechaza si es el último administrador.
SET_PASSWORD	SET_PASSWORD usuario contraseña	Cambia la contraseña de un usuario existente.
HELP	HELP [comando]	Lista los comandos disponibles o describe uno en particular.
QUIT	QUIT	Cierra la sesión.

Los errores utilizan mensajes específicos:

- -ERR syntax error
- -ERR unknown command
- -ERR unknown param
- -ERR invalid value
- -ERR user exists
- -ERR user not found
- -ERR user table full
- -ERR cannot delete last admin
- -ERR not authenticated
- -ERR already authenticated
- -ERR line too long

2.2.6 Parámetros configurables

El comando CONFIG acepta los siguientes parámetros, con validación de rango en el servidor:

Parámetro	Efecto	Rango válido
timeout	Segundos de inactividad tras los cuales se cierra una sesión SOCKS (0 = deshabilitado)	0 – 86400
max_connections	Tope de clientes SOCKS simultáneos	1 – 65535
io_buffer_size	Tamaño de buffer de E/S para sesiones nuevas	1024 – 65536

2.2.7 Pipelining

El servidor soporta *pipelining*: un cliente puede enviar varios comandos en un mismo segmento TCP (o incluso un comando fragmentado en varios segmentos) y el comportamiento es idéntico al de recibirlos de a uno. Esto se logra acumulando los bytes recibidos en un buffer y alimentando el intérprete de líneas byte a byte, sin asumir jamás que una lectura del socket corresponde a un mensaje completo.

2.2.8 Cierre de la conexión

Existen dos formas de terminar una sesión:

1. **QUIT**: El servidor encola +OK y marca la sesión para cierre; el descriptor se cierra recién cuando el buffer de salida se vació, garantizando que el cliente reciba todas las respuestas pendientes.
2. **Half-close**: El cliente puede cerrar su lado de escritura después de enviar sus comandos. Al detectar el fin de archivo, el servidor procesa todos los comandos pendientes en el buffer, incluida una eventual última línea sin \n terminador, envía las respuestas y recién entonces cierra la conexión.

2.2.9 Límites propuestos

Límite	Valor
Longitud máxima de línea	511 bytes
Longitud máxima de nombre de usuario	64 caracteres
Longitud máxima de contraseña	255 caracteres
Cantidad máxima de usuarios	256
Capacidad del registro de accesos	4096 entradas (buffer circular: las más antiguas se sobrescriben)
Longitud máxima de host de destino registrado	255 caracteres

2.3 Aplicación Cliente

Para cumplir con el requisito de brindar una forma cómoda de administrar el servidor desde la terminal se desarrolló bin/client, una aplicación de línea de comandos que implementa un REPL (Read Evaluate Print Loop) interactivo sobre el protocolo ChungusMonitor. Dada la simpleza del intercambio, el cliente utiliza entrada/salida bloqueante, lo cual está permitido por la consigna.

La invocación general es:

```
./bin/client -p <puerto>
```

El cliente se conecta al servidor, recibe el saludo y entra en un bucle interactivo donde el operador puede ingresar comandos del protocolo (AUTH, STATS, USERS, etc.) línea por línea. Cada comando se envía al servidor al presionar Enter y la respuesta se lee hasta el delimitador “.” (descrito en 2.2.3). El comando QUIT cierra la sesión y termina el programa. HELP puede ejecutarse sin autenticación previa, en concordancia con el protocolo.

Tras enviar el comando, el cliente lee la respuesta línea por línea hasta encontrar el delimitador . (descrito en 2.2.3) y la muestra en pantalla. Luego vuelve al bucle de lectura para el próximo comando. El comando QUIT cierra la sesión y termina el programa. HELP puede ejecutarse sin autenticación previa.

3. Proceso de Desarrollo

El proyecto se abordó de manera incremental: cada etapa produjo un servidor funcional y verificable antes de pasar a la siguiente. Esta estrategia, recomendada por la cátedra, evitó el riesgo de implementar primero una versión bloqueante del proxy completo y tener que cambiarla luego al modelo no bloqueante — una migración notoriamente costosa. Los módulos provistos por la cátedra (selector, buffer, máquina de estados y parser) se integraron desde el comienzo y estructuraron todo el diseño posterior.

3.1 Echo Server Bloqueante

La primera etapa consistió en un servidor de eco elemental: un socket pasivo TCP que acepta una conexión, lee datos y los devuelve. Su propósito fue afianzar el manejo de la API de sockets de Berkeley (socket, bind, listen, accept, read, write) y de condiciones como el fin de archivo (read == 0). Esta versión, deliberadamente bloqueante, sirvió como punto de partida conceptual y no forma parte del producto final.

3.2 Echo Server No Bloqueante

El paso decisivo fue reescribir el eco sobre el selector de la cátedra: un único hilo que multiplexa múltiples descriptores y despacha manejadores de lectura y escritura cuando el kernel indica que una operación no bloqueará. Cada conexión pasó a tener su propio buffer, y los intereses de cada socket (lectura si hay espacio en el buffer, escritura si hay datos pendientes) se recalculan tras cada evento. Esta versión, conservada en `src/server/echo/echo.c` con sus pruebas automatizadas, estableció el patrón estructural — socket pasivo registrado en el selector, sesión con buffer propio, manejadores por descriptor — que se replicó luego tanto en el proxy SOCKS como en el servidor de monitoreo.

3.3 Implementación de SOCKSv5

Sobre esa base se construyó el proxy en fases sucesivas. Primero se incorporó la conexión saliente no bloqueante hacia un origen y el camino de escritura con buffers por socket, manejando escrituras parciales. Luego se implementaron los tres parsers del protocolo — negociación de métodos, autenticación RFC 1929 y request CONNECT — como módulos independientes sin entrada/salida, cada uno con sus pruebas unitarias, lo que permitió validarlos alimentándolos byte a byte sin necesidad de sockets. La máquina de estados de la cátedra integró estas piezas: cada estado del protocolo tiene sus manejadores y las transiciones se derivan del resultado de los parsers. Finalmente se agregaron la resolución DNS en un hilo auxiliar con notificación al selector, el failover entre direcciones y los códigos de error específicos. La validación de extremo a extremo se realizó con:

```
curl --socks5 admin:admin@localhost:1080 http://example.com
```

3.4 Desarrollo del Protocolo de Administración y Aplicación Cliente

Con el proxy operativo se diseñó el protocolo ChungusMonitor (sección 2.2) y se implementó en dos capas deliberadamente separadas: **monitor_commands.c** contiene el intérprete de líneas y los manejadores de comandos **sin ningún socket** — recibe bytes y escribe respuestas en un buffer —, mientras que `monitor.c` aporta la capa de red (socket pasivo, sesiones, manejadores del selector). Esta separación permitió probar toda la lógica del protocolo de forma unitaria, incluyendo pipelining y half-close, sin depender de la red. El estado compartido se centralizó en el módulo store (`monitor_store`): la autenticación SOCKS y los comandos de administración operan sobre la misma tabla de usuarios, y el proxy reporta allí métricas, sesiones y accesos. Por último se desarrolló la aplicación cliente (sección 2.3) y se completó el apagado ordenado del servidor ante SIGINT y SIGTERM.

4. Problemas Encontrados

Durante el desarrollo surgió la dificultad de mapear fallos reales de red a los códigos de respuesta (REP) definidos por el RFC 1928. El estándar enumera ocho valores distintos —desde éxito (**0x00**) hasta tipos de dirección no soportados (**0x08**)—, pero en la práctica de un proxy que solo admite **CONNECT** sobre TCP la mayoría son poco frecuentes o directamente inaplicables. El código **0x06** (*TTL expired*), por ejemplo, está pensado para el reenvío de datagramas en **UDP ASSOCIATE** y no aparece en escenarios de conexión saliente TCP; documentarlo en el RFC no implica que un cliente SOCKS habitual lo reciba. Ante esta disparidad entre el catálogo del estándar y el tráfico real, se priorizaron los casos verificables con herramientas estándar: **0x04** cuando falla la resolución DNS, **0x05** cuando ninguna dirección devuelta acepta la conexión, y **0x01** para los demás fallos (mensajes mal formados, **BIND/UDP** solicitados, falta de recursos). Comandos rechazados o direcciones no admitidas podrían devolver **0x07** u **0x08** con mayor fidelidad al RFC, pero en las pruebas funcionales con *curl* esos casos se resuelven igualmente con el código genérico.

Otro punto de confusión fue la distinción entre el parámetro **timeout** configurable en ChungusMonitor y el **TTL** de registros DNS. El comando **CONFIG timeout** mide inactividad de una sesión SOCKS ya establecida (por defecto deshabilitado con valor 0) y se evalúa comparando la última actividad con la hora actual; no influye en la resolución de nombres ni reimplementa la caducidad de entradas en caché. Dado que cada FQDN se resuelve con **getaddrinfo** en el momento del pedido —sin caché intermedia—, el TTL de los registros lo gestiona únicamente el resolvidor del sistema operativo y no constituyó un problema de implementación en sí.

Por último, la interacción entre E/S no bloqueante, **connect()** asíncronico y failover entre direcciones exigió un cuidado extra al propagar errores: un fallo inmediato en una entrada de **addrinfo** no es equivalente al rechazo reportado por **getsockopt(SO_ERROR)** tras un evento de escritura en el socket de origen, y ambos debían converger en el mismo mecanismo de reintento sin bloquear el hilo del selector. La resolución DNS, al ser la única operación bloqueante permitida, reforzó la separación entre lógica de protocolo (parsers sin E/S) y manejo de red: los parsers se probaron alimentándolos byte a byte, incluido pipelining de greeting, autenticación y request en un solo segmento, antes de integrar la máquina de estados completa.

5. Limitaciones

La primera limitación del proyecto concierne a la cantidad máxima de conexiones SOCKS simultáneas. Ese tope lo fija el parámetro **max_connections** del **monitor_store** (1024 por defecto), comprobado al aceptar cada cliente: si el cupo está lleno, la conexión entrante se cierra sin registrarla en el selector. En fase de relay, cada sesión activa utiliza dos descriptors registrados en el multiplexor (socket del cliente y socket hacia el origen), además de los sockets pasivos de SOCKS y monitoreo y de las conexiones de administración. El selector se crea con **selector_new(1024)** como pre-dimensionado inicial de su tabla interna y del buffer de eventos de **epoll**, pero puede crecer dinámicamente al registrar descriptors de número más alto; no constituye, por sí solo, el límite principal de concurrencia. Complementariamente, el arreglo **sessions[]** —donde se rastrean las sesiones para métricas, el comando **CONNECTIONS** y el registro de accesos— se dimensiona una sola vez al arrancar (**sessions_cap = max_connections** inicial). Elevar **max_connections** en tiempo de ejecución mediante **CONFIG** actualiza el contador lógico, pero no redimensiona ese arreglo: el cupo real de sesiones rastreadas no puede superar el valor con el que se inició el servidor.

Otra limitación importante es que no se implementó el soporte para los comandos **BIND** ni **UDP ASSOCIATE** del protocolo SOCKSv5. Solo se admite **CONNECT**. Se decidió dejar fuera del alcance la gestión de sockets UDP asociados, el reenvío de datagramas sin conexión previa y las asociaciones de escucha exigidas por **BIND**, por la complejidad que agregan y porque no son requisitos centrales de la consigna.

Por último, la tabla de usuarios (hasta 256 entradas), las métricas y el registro de accesos (buffer circular de 4096 entradas) residen exclusivamente en memoria: un reinicio del servidor los pierde y restablece el usuario inicial **admin/admin**. La consigna admite métricas volátiles, pero la falta de persistencia de usuarios y la rotación del log constituyen limitaciones operativas. Tampoco se dispone en el repositorio de pruebas de estrés ni de mediciones empíricas de rendimiento con 500 o más clientes concurrentes.

6. Posibles Extensiones

En el plano funcional del proxy SOCKS quedan pendientes los comandos **UDP ASSOCIATE** y **BIND** del RFC 1928, que exigirían extender la máquina de estados, hoy orientada a TCP, con relay de datagramas o escucha de conexiones entrantes hacia el cliente. El estándar también define el método de autenticación **GSSAPI (0x01)**, la consigna del TPE lo menciona en el RFC 1928 pero aclara explícitamente que no es un requisito de entrega. En la implementación actual, el parser de greeting solo reconoce **0x02** (usuario/contraseña, RFC 1929) y **0x00** (sin autenticación): si el cliente ofrece ambos, se prioriza **0x02** y las credenciales se validan contra el **monitorstore**; si solo ofrece **0x00**, el servidor acepta y registra la sesión como usuario **Anon** antes de pasar al request **CONNECT**. Cualquier otro método, incluido **0x01**, se ignora en la negociación y, si no queda ninguna opción compatible, el servidor responde **05FF** y cierra la conexión. Incorporar GSSAPI implicaría extender el greeting para seleccionarlo, añadir un nuevo estado en la máquina de estados con la sub negociación GSS-API (un intercambio de tokens que puede ser de varios mensajes) y, probablemente, delegar las llamadas bloqueantes a la librería GSS en un thread auxiliar con el mismo esquema usado hoy para **getaddrinfo**, sin reutilizar la tabla de usuarios de ChungusMonitor. Por esa complejidad y porque el objetivo del trabajo es demostrar autenticación usuario/contraseña, GSSAPI queda como extensión orientada a despliegues

A nivel operativo, convendría persistir en disco la tabla de usuarios y el **ACCESSLOG** (carga al arrancar y volcado atómico tras **ADD_USER**, **DEL_USER** y **SET_PASSWORD**), almacenar contraseñas como hashes en lugar de texto plano, y gobernar el acceso anónimo desde ChungusMonitor. En DNS, **getaddrinfo_a(3)** podría reemplazar el hilo auxiliar por consulta, complementado con un caché con TTL configurable.

ChungusMonitor podría ampliarse con métricas adicionales (estadísticas DNS, desglose por usuario o destino, **GET_CONFIG**, exportación del log) y el cliente con sub comandos más cómodos. Para demostrar el requisito de ≥ 500 clientes concurrentes habría que atender dos cuellos de botella del diseño actual: el selector se inicializa con **selector_new(1024)** y cada sesión en relay registra dos descriptores (cliente y origen), además de los sockets pasivos y las conexiones de administración, de modo que el tope real queda por debajo del valor nominal de **maxconnections**; y el array **sessions[]** de **monitorstore** se asigna una sola vez al arrancar, por lo que elevar **maxconnections** mediante **CONFIG** actualiza el contador pero no redimensiona **sessions_cap**. Las pruebas de estrés que documenten throughput y degradación deberían ejecutarse además con un build **release** (quitando **-g** y sanitizers de **makefile.inc**), para no distorsionar las mediciones. Por último, la segunda entrega opcional plantea inspeccionar el tráfico ya tunelizado de forma análoga a ettercap para registrar credenciales **POP3** enviadas en

texto claro: durante el relay los bytes circulan por los buffers **c2o** y **o2c**, donde un analizador podría detectar los comandos **USER** y **PASS**, guardar el par usuario/contraseña en el store y exponerlo al administrador mediante un comando nuevo de ChungusMonitor, sin alterar el flujo SOCKS.

7. Conclusiones

Como conclusión del proyecto, se pudo desarrollar un servidor proxy SOCKSv5 completo junto con un protocolo de administración propio y su correspondiente cliente, partiendo de un servidor de eco básico. Este proceso permitió profundizar en la comprensión de conceptos fundamentales relacionados con protocolos de red, autenticación, resolución de nombres, manejo de errores y comunicación mediante entrada/salida no bloqueante.

Por otro lado, se destaca que la principal restricción de la consigna, consistente en implementar el servidor utilizando un único hilo con entrada/salida no bloqueante multiplexada, tuvo una influencia determinante en el diseño de la solución. Esta condición llevó a adoptar una arquitectura basada en eventos, con una clara separación entre la lógica de los protocolos, implementada mediante parsers y máquinas de estado, y el manejo de la entrada/salida, encargado de la transferencia de datos entre los distintos extremos de la comunicación.

Por último, la estrategia de desarrollo incremental resultó fundamental para la evolución del proyecto. Este enfoque permitió mantener una aplicación funcional durante todo el proceso de implementación, facilitar la incorporación progresiva de nuevas funcionalidades y desarrollar pruebas automatizadas para los componentes más críticos. Asimismo, la reutilización de la misma estructura de diseño en los servidores de eco, SOCKSv5 y monitoreo permitió obtener una solución más consistente, modular y fácil de mantener.

8. Guía de Instalación

8.1 Requisitos del Entorno

El proyecto está pensado para compilar y ejecutar en Linux . Las dependencias son:

```
sudo apt-get update

sudo apt-get install -y build-essential gcc make check
pkg-config
```

La biblioteca Check solo es necesaria para compilar y correr las pruebas automatizadas; en Ubuntu 24.04 el paquete se llama *check*.

8.2 Obtención del Código y Compilación

```
git clone https://github.com/SebaNLee/Protos-TPE-20261Q.git

cd Protos-TPE-20261Q

make clean all
```

Comandos disponibles del *makefile*:

Comando	Descripción
make all	Compila bin/server y bin/client
make server / make client	Compila un binario individual
make test	Compila las pruebas en build/
make check	Compila y ejecuta todas las pruebas
make clean	Elimina obj/, bin/ y build/

Dado que Make no rastrea dependencias de encabezados, tras modificar un archivo .h corresponde ejecutar:

```
make clean && make all
```

8.3 Entorno Docker

Para desarrollo en sistemas no Linux (por ejemplo macOS) el repositorio incluye un Dockerfile (Ubuntu 24.04, plataforma linux/amd64) y un docker-compose.yml que publica los puertos 1080 y 8080:

```
docker compose build

docker compose run --rm dev make clean all test check

docker compose run --rm --service-ports dev ./bin/server
```

8.4 Artefactos Generados

Ruta	Contenido
bin/server	Servidor proxy SOCKSv5 + monitoreo
bin/client	Cliente de monitoreo de línea de comandos
build/*_test	Binarios de pruebas
obj/	Objetos intermedios de compilación

9. Guía de Configuración

El servidor puede configurarse, mediante distintos flags al momento de su ejecución.

9.1 Argumentos del Servidor

```
./bin/server [-p <puerto_socks>] [-m <puerto_monitor>] [-h]
```

Flag	Descripción	Valor por defecto
-p	Puerto de escucha del servicio SOCKSv5	1080
-m	Puerto de escucha del servicio de monitoreo	8080
-h	Imprime el mensaje de uso y termina	—

Ambos sockets pasivos son IPv6 *dual-stack*, por lo que aceptan clientes IPv4 e IPv6 sin configuración adicional.

9.2 Argumentos del Cliente de Monitoreo

```
./bin/client -p <puerto> [-u <usuario>] [-w <contraseña>] [-h]  
<comando> [args...]
```

Flag	Descripción	Valor por defecto
-p	Puerto del servicio de monitoreo	8080
-u	Usuario administrador	admin
-w	Contraseña	admin
-h	Imprime el mensaje de uso y termina	—

Ambas aplicaciones procesan sus argumentos con `getopt(3)`, siguiendo las convenciones de utilidades de POSIX.

9.3 Parámetros en Tiempo de Ejecución

La configuración dinámica se realiza a través del comando CONFIG del protocolo de monitoreo (o del subcomando config del cliente), sin reiniciar el servidor:

Parámetro	Descripción	Por defecto	Rango
timeout	Segundos de inactividad tras los cuales se cierra una sesión SOCKS; 0 la deshabilita	0	0 – 86400
max_connections	Máximo de sesiones SOCKS simultáneas (las excedentes se rechazan al aceptar)	1024	1 – 65535
io_buffer_size	Tamaño de buffer de E/S para sesiones nuevas (véase la limitación en 5.2)	4096	1024 – 65536

9.4 Usuario Inicial y Roles

Al arrancar, el servidor crea el usuario **admin** con contraseña **admin** y rol administrador. Existen dos roles:

- **user**: puede autenticarse en el proxy SOCKS.
- **admin**: además puede autenticarse en el puerto de monitoreo y administrar el servidor.

Los usuarios se gestionan en tiempo de ejecución con **add-user**, **del-user** y **set-password**. El sistema impide eliminar al último administrador, garantizando que el servidor nunca quede sin acceso de administración.

10. Ejemplos de Prueba

10.1 Pruebas Automatizadas

El proyecto cuenta con binarios de prueba basados en la biblioteca Check, que se compilan y ejecutan con ***make check***:

Prueba	Alcance
buffer_test, selector_test, parser_test, parser_utils_test, netutils_test, stm_test	Módulos compartidos de la cátedra
echo_test	Servidor de echo no bloqueante
greeting_test, auth_test, request_test	Parsers SOCKS (negociación, RFC 1929, CONNECT con IPv4/FQDN y comandos inválidos)
store_test	Estado compartido entre usuarios, configuración, sesiones y registro de accesos
monitor_commands_test	Intérprete de comandos ChungusMonitor (sintaxis, autenticación y errores)
monitor_test	Integración TCP del monitoreo: saludo, autenticación, pipelining, half-close y alta de usuarios

10.2 Pruebas Funcionales del Proxy SOCKS

Con el servidor en ejecución, el funcionamiento se verifica con curl usando el proxy:

Destino por FQDN (resolución DNS en el servidor)

```
curl --socks5-hostname admin:admin@localhost:1080 http://example.com
```

```
MacBook-Air-8:~ geronimo$ curl --socks5-hostname admin:admin@localhost:1080 http://example.com
<!doctype html><html lang="en"><head><title>Example Domain</title><link rel="icon" href="data:,"><meta name="viewport" c
ontent="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-
ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domai
n</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><p><a
href="https://iana.org/domains/example">Learn more</a></p></div></body></html>
MacBook-Air-8:~ geronimo$
```

Destino por FQDN resuelto por curl (el proxy recibe la IP)

```
curl --socks5 admin:admin@localhost:1080 http://example.com
```

```
MacBook-Air-8:~ geronimo$ curl --socks5 admin:admin@localhost:1080 http://example.com
curl: (97) Could not resolve host: example.com
MacBook-Air-8:~ geronimo$
```

Credenciales inválidas: curl reporta el rechazo de autenticación

```
curl --socks5 admin:incorrecta@localhost:1080 http://example.com
```

```
MacBook-Air-8:~ geronimo$ curl --socks5 admin:incorrecta@localhost:1080 http://example.com
curl: (97) User was rejected by the SOCKS5 server (1 1).
MacBook-Air-8:~ geronimo$
```

Destino inexistente: el proxy responde host inalcanzable (REP 0x04)

```
curl --socks5-hostname admin:admin@localhost:1080
http://dominio-inexistente.invalid
```

```
MacBook-Air-8:~ geronimo$ curl --socks5-hostname admin:admin@localhost:1080 http://dominio-inexistente.invalid
curl: (97) Can't complete SOCKS5 connection to dominio-inexistente.invalid. (4)
MacBook-Air-8:~ geronimo$
```

Puerto cerrado: el proxy responde conexión rechazada (REP 0x05)

```
curl --socks5 admin:admin@localhost:1080 http://localhost:1
```

```
MacBook-Air-8:~ geronimo$ curl --socks5 admin:admin@localhost:1080 http://localhost:1
curl: (97) Can't complete SOCKS5 connection to localhost. (5)
MacBook-Air-8:~ geronimo$
```

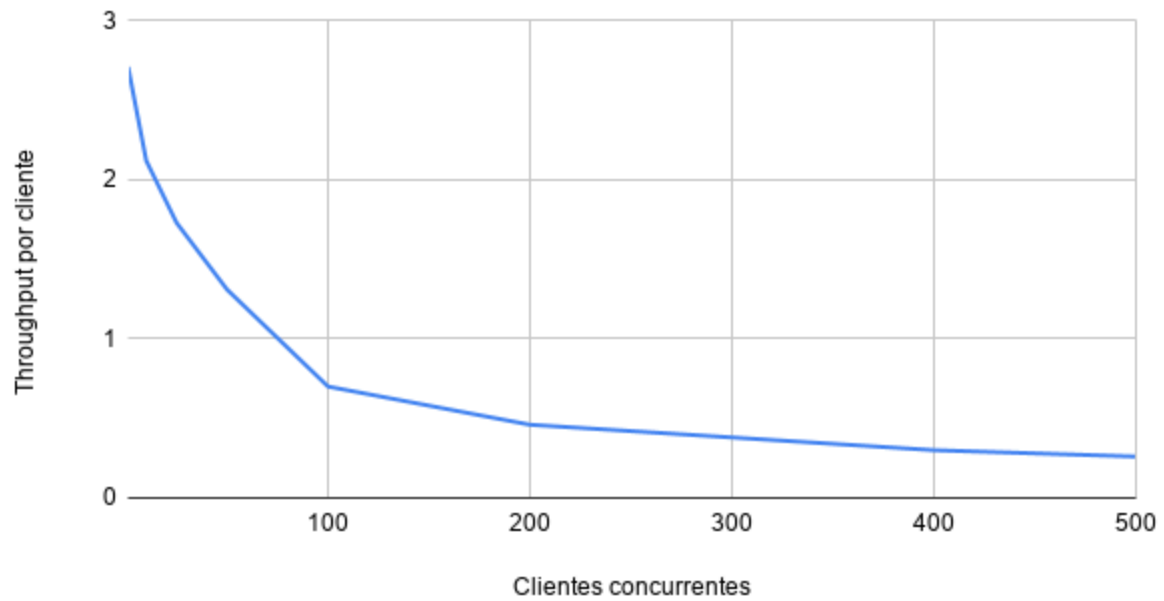
10.3 Pruebas de Estrés y Throughput

Para evaluar el rendimiento se desarrolló un generador de carga en C (*stress_client*) que implementa el handshake SOCKS5 completo. El modo *connections* abre N túneles hacia un *echo_backend* local y los mantiene idle 1 s antes de cerrarlos. El modo *throughput* transfiere 64 KiB por cliente en ping-pong de 4 KiB (write/read por chunk) para evitar deadlock en el túnel. El modo *live* mantiene túneles abiertos mientras el operador consulta el monitor (*bin/client: STATS, CONNECTIONS*) en otra terminal.

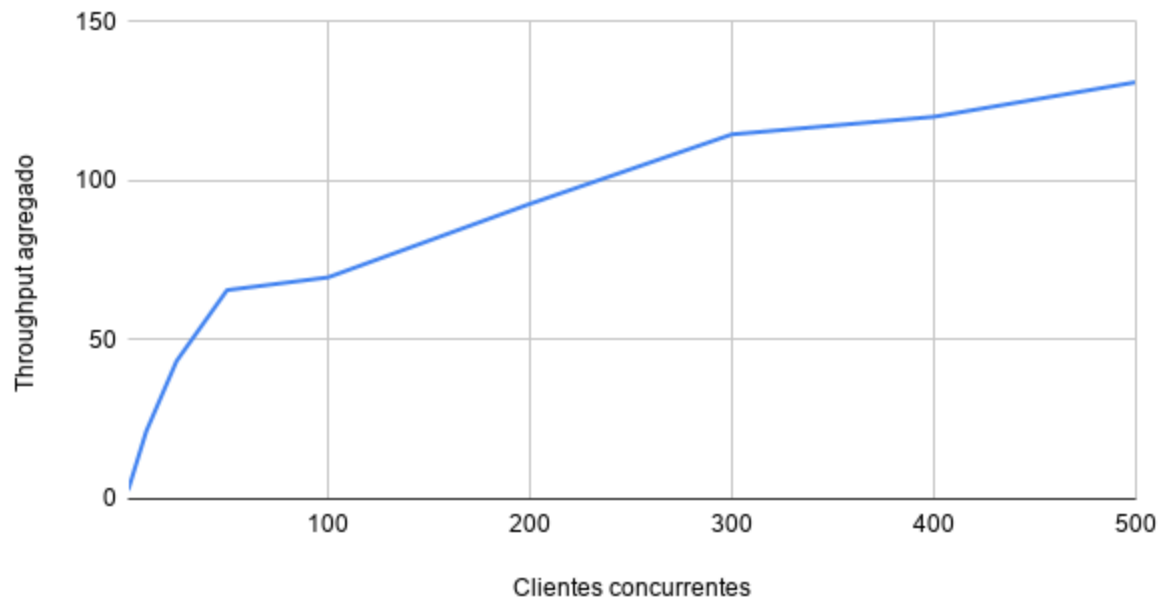
En escalabilidad (default *max_connections=1024*), se probó de 10 a 600 túneles sin fallos; el tiempo creció ~linealmente (~1 s para 10, ~10,4 s para 600). Con *CONFIG max_connections 16384*, una prueba puntual alcanzó 16.384/16.384 conexiones simultáneas en ~267 s (tope arquitectónico 2^{14}).

En throughput, el caudal agregado sube hasta ~100–118 MiB/s (estabilizado ~100+ MiB/s con $N \geq 100$); el caudal por cliente baja (~3,0 MiB/s con $N=1$, ~0,24 MiB/s con $N=500$), coherente con un servidor single-thread multiplexado con *epoll*. No hubo errores ni pérdida de datos en las corridas documentadas.

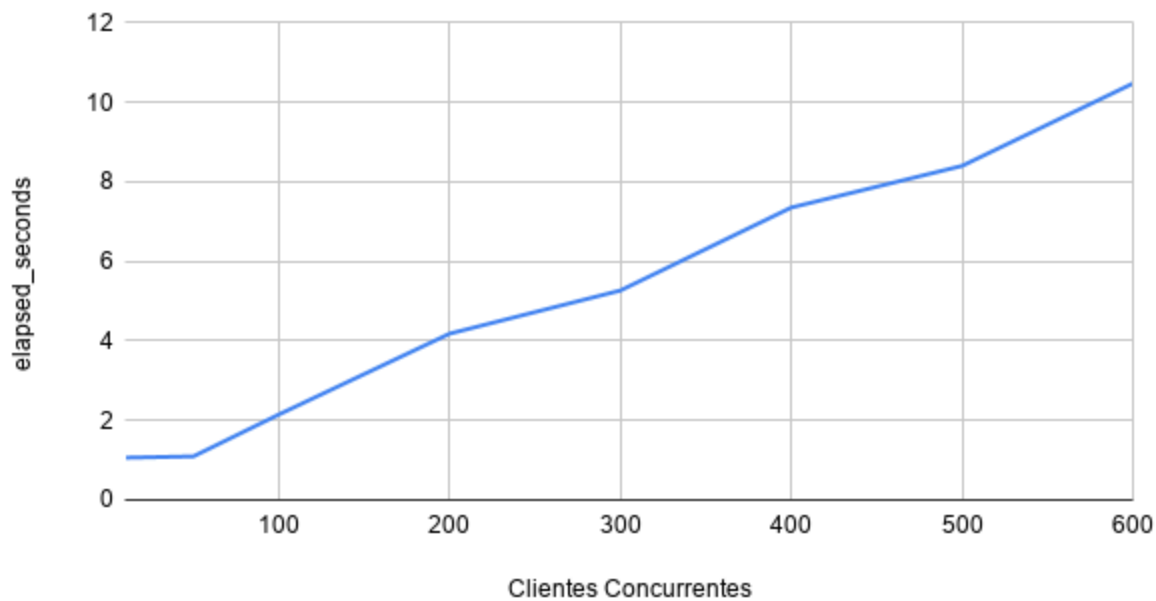
Clientes concurrentes vs Throughput por cliente



Clientes concurrentes vs Throughput agregado



Cientes concurrentes vs Segundos Transcurridos



11. Ejemplos de Configuración y Monitoreo

Los siguientes ejemplos muestran sesiones reales de administración utilizando la aplicación cliente. En cada caso se indica el comando ejecutado y la salida que produce (las cifras dependen del estado del servidor).

Gestion de usuarios

```
+OK Hello! from Proxy/1.0
AUTH admin admin
+OK
```

Figura X: Saludo del servidor y autenticación del usuario

```
ADD_USER alice pass_secreta123
+OK
```

Figura X: Agregar un usuario nuevo

```
SET_PASSWORD alice nuevacleve
+OK
```

Figura X: Cambiar la contraseña de un usuario existente

```
ADD_USER operador clave456 admin
+OK
```

Figura X: Crear un usuario con privilegios de administrador

```
DEL_USER operador
-ERR cannot delete last admin
```

Figura X: Intentar eliminar el administrador más reciente

Registro de Accesos y Sesiones Activas

Los siguientes resultados se realizaron con dos usuarios activos en simultáneo.

```
USERS
+OK operador
+OK admin
```

Figura X: Usuarios activos conectados al servidor.

```
CONNECTIONS
+OK admin httpbin.org:80 RELAY 83 0
+OK operador httpbin.org:80 RELAY 83 0
```

Figura X: Conexiones activas.

```
STATS
+OK total_connections=34 concurrent_connections=2 bytes_up=1707 bytes_down=15092
```

Figura X: Estadísticas del servidor.

Cambios de Configuración en Tiempo de Ejecución

```
CONFIG timeout 300
+OK
CONFIG max_connections 512
+OK
CONFIG io_buffer_size 8192
+OK
CONFIG timeout 999999
-ERR invalid value
```

Figura X: Cambios de configuración

12. Documento de Diseño del Proyecto

El sistema es un único proceso con un único hilo principal. Dos sockets pasivos, SOCKS y monitoreo, conviven en el mismo selector; cada conexión aceptada registra su propio descriptor con manejadores específicos del servicio. El único punto donde aparecen hilos adicionales es la resolución DNS, y su única interacción con el resto del sistema es depositar el resultado y despertar al selector.

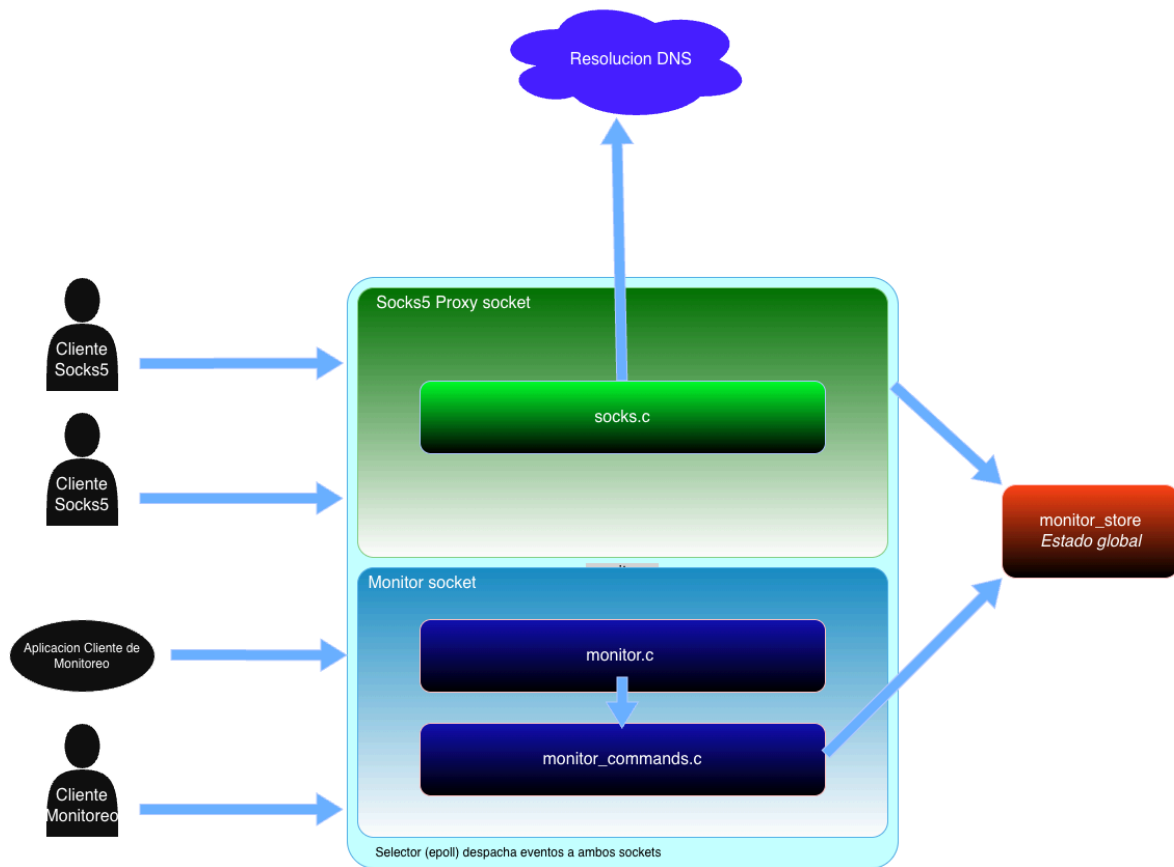


Figura X: Diagrama de diseño del proyecto

13. Bibliografía

1. RFC 1928 — *SOCKS Protocol Version 5*. <https://datatracker.ietf.org/doc/html/rfc1928>
2. RFC 1929 — *Username/Password Authentication for SOCKS V5*.
<https://datatracker.ietf.org/doc/html/rfc1929>